

Programming Assignment 4: Evil Hangman

Due: Thursday, February 26th at 11pm CT

Learning Goals

- To understand how to use maps, lists, and sets as a client
- To design and implement a complex program using multiple data structures

Assignment Guidelines and Specifications

This is an individual assignment, which means you cannot work with a partner on this assignment. All code submitted must be entirely your own.

For this assignment, there are some additional specifications:

- You may use any classes and methods from the Java standard library that you choose.
 - **You must use a `Map<String, <ArrayList<String>>` to track patterns and their associated word families.**
- You may (and are encouraged to) create private helper methods to remove redundancy and provide structure to your solution.
- You *may* create and share tests with other students. We encourage you to use Ed to share tests with other people.

Introduction

You may already be familiar with the game Hangman (you can read about it [here](#) or try [this](#) online version). In this assignment, you will be creating *Evil Hangman*, a cruel version of the original game where your opponent, the computer, very cleverly cheats.

Getting Started

Here are the implementation files for this assignment:

- [HangmanManager.java](#): This file contains the class that keeps track of the state of the Evil Hangman game. You need to complete this file according to the given specifications and submit this file.
- [HangmanMain.java](#): This file contains the main driver class for this assignment. You should **not** modify this file and do not need to submit this file.
- [HangmanDifficulty.java](#): This file contains the enumerated type with difficulty levels for the game. It is used in HangmanMain and HangmanManager. You should **not** modify this file and do not need to submit this file.

Here are the dictionaries you should use to test your code:

- [smallDictionary.txt](#): This dictionary has just 9 words, which makes it good for testing. These are the same words used in the examples in this document. The code in HangmanMain currently uses this dictionary.
- [dictionary.txt](#): This dictionary has thousands of words. You should change the code in HangmanMain to test with this dictionary as well.

Here are some sample program outputs. Your program's output must match the sample runs without debugging **exactly**. The only allowable difference is when a player loses (the word printed is random and so the outputs may not match). Use a [diffchecker](#) to compare our program's output to your program's output (do not compare the outputs by hand, you will not be able to find all the differences).

- [Sample Log 1](#): using smallDictionary.txt, with debugging, hardest difficulty
- [Sample Log 2](#): using smallDictionary.txt, with debugging, easiest difficulty
- [Sample Log 3](#): using smallDictionary.txt, without debugging, various difficulties
- [Sample Log 4](#): using dictionary.txt, with debugging, hardest difficulty
- [Sample Log 5](#): using dictionary.txt, with debugging, easiest difficulty
- [Sample Log 6](#): using dictionary.txt, without debugging, hardest difficulty
- [Sample Log 7](#): using dictionary.txt, without debugging, medium difficulty
- [Sample Log 8](#): using dictionary.txt, without debugging, easiest difficulty

To help test your code, we have provided you with access to our testing program and the kinds of tests we run. Of course, the actual tests we use will be different, but this is still a great resource:

- [EvilHangmanAutoTester.java](#): This tester file uses your HangmanManager and the tests encoded in evilTests_STUDENT_VERSION.eht to automatically test your code. As new test files get generated, you can alter the name of the input file as necessary.
- [evilTest_STUDENT_VERSION.eht](#): This is the data file for the EvilHangmanAutoTester. This file is not human readable.
- [GenerateTests.java](#): You can use this class if you want to generate your own tests. Replace the test methods or add new ones and change the name of the output file. This class relies on a data file and your HangmanManager to generate tests. Here is an example file, [studentTestData.txt](#), which was used to generate evilTest_STUDENT_VERSION.eht referenced above.

Here is some useful Java documentation that you should reference:

- Java API for [Map Interface](#), [TreeMap](#), and [HashMap](#)
- Java API for [ArrayList](#)
- Java API for [Set Interface](#)
- Java API for [StringBuilder](#)

Evil Hangman

Evil Hangman is, well, *evil*, because the computer puts off actually picking a word for as long as possible. The program starts off with a dictionary (list) of words (strings). As the user guesses letters, the computer narrows the list of words that are *active*, or words that could still be chosen given the user's guesses. The words in the active list all have the same *pattern* as the "word" displayed to the user. In other words, the program does not pick the secret word until it has to. Instead, it keeps a list of possible words based on user guesses.

A *pattern* consists of the letters that have been guessed correctly and the letters that have not been guessed yet. For example, if the current pattern is "**-e-e-**" then the words "beget", "beret", "bevel", "defer", "deter" and so forth would be in the list of active words. Those words match the pattern. The "**-**" characters are wildcard characters and represent any character that has not been guessed yet.

When the user guesses a letter the computer splits the current list of words into different *families* (various lists of words) based on the letter the user just guessed and the resulting pattern of that letter in words from the current list. The computer then chooses a single list of words from every possible list of words to become the *active* list (the list to continue the game with, or in other words, the list of words that could potentially be the secret word). If the computer is being evil, it picks the hardest list of words as the new active list. In general, the longest list of words is considered the hardest, but there are tiebreakers explained below. The program also allows the difficulty to vary so the picked list may not always be the list with the largest number of words.

There is an in-depth explanation of how the program cheats in the section below titled "How the Program Cheats." You should read this explanation before implementing any of the methods.

HangmanManager

Complete the HangmanManager class, which keeps track of the state of the game and narrows the list of words as the user makes guesses. **You must use a map to keep track of the patterns and associated list of words that match that pattern.** You should add any instance variables you need to the class, but don't add any that are unnecessary or redundant. Add helper methods as necessary to keep methods small and easy to understand. It is easy to overcomplicate this assignment.

For reference, our solution uses 8 instance variables, 5 private helper methods, and is around 315 lines long (including blank lines, debugging code, method header comments, imports, comments, and lines with single closing braces). Our solution has around 170 lines of "actual" code.

Here are the methods you will need to implement. You should not necessarily implement them in this order.

1. Implement the constructors:

```
public HangmanManager(Set<String> words, boolean debug)

public HangmanManager(Set<String> words)
```

These constructors will create a new `HangmanManager` from the provided set of words and phrases. `words` will be a non-empty `Set` of `Strings`. You can assume that these `Strings` will only contain lowercase English letters.

For the second constructor (with only a `Set` parameter), debugging mode should be turned off.

These constructors will create a copy of the dictionary. You may also want them to perform some other work, depending on your implementation of `HangmanManager`. Note that the majority of the preparation work for the game should be done in `prepForRound`.

2. Implement the following method:

```
public void prepForRound(int wordLen, int numGuesses, HangmanDifficulty diff)
```

The `HangmanMain` program allows the user to play the game. Instead of creating a new `HangmanManager` object each time a person plays a game, this method prepares (or "resets") the `HangmanManager` object with the new word length, number of wrong guesses allowed, and difficulty. This method must also reset the list of eligible active words based on the original dictionary, the pattern, and the number of wrong guesses. Note that each round specifies a word length. You should only make words of that length eligible for this round.

This method must be called before each game, including the first one.

3. Implement the following method:

```
public TreeMap<String, Integer> makeGuess(char guess)
```

This method does the heavy lifting of your program!

The precondition for this method is that the `char` guess has not already been guessed. If the character has already been guessed, throw an `IllegalStateException`.

Using this guess, the `HangmanManager` decides what set of words to use going forward. The method returns a map of the patterns it generated and how many words matched each pattern. Note that the `HangmanManager` does not keep track of this map as an instance variable. After this method, the active list of words may be changed.

You must create and use a local variable of type `Map<String, ArrayList<String>>` to break the active word list up into new families of words. There is a detailed explanation on how to do this further in the document.

The `String` in the `Map` returned represents the possible word patterns based on the guess and the list of active words. The values are the number of words that fit this pattern. The sample output with debugging on shows exactly what this map should be for the given conditions. We suggest putting off building this second map returned by the method until the end of the method. This return value is used for debugging and testing purposes; it's not actually used by anything else in the class.

There is much to do in this method including updating the active pattern, the letters that have been guessed, and the number of wrong guesses if the pattern doesn't change.

Part of this method includes picking the pattern with the correct difficulty level. You can find more information about how to do this in the "Choosing Difficulty" section below.

4. Implement the following method:

```
public boolean alreadyGuessed(char guess)
```

This method returns true if the user has already guessed the given character, false otherwise.

5. Implement the following method:

```
public int getGuessesLeft()
```

This method returns the number of wrong guesses the user has left.

6. Implement the following method:

```
public String getGuessesMade()
```

This method returns the letters the user has guessed so far, in sorted order.

7. Implement the following method:

```
public String getPattern()
```

This method returns the current pattern to be displayed for the current Evil Hangman game, taking into account guesses that have been made. Letters that have not yet been guessed should be displayed as a dashes. There should **only** be lowercase letters and dashes in the returned `String`.

8. Implement the following method:

```
public String getSecretWord()
```

This method gets the secret word the computer finally settles on. If the active list of words has more than one element, then pick a word arbitrarily (at random) from the active list. If the list has been narrowed to one word, return that word.

This method throws an `IllegalStateException` if the list of active words is size 0 (this should never occur if you implement the program correctly).

9. Implement the following method:

```
public int numWords(int length)
```

This method returns the number of words in the original dictionary of the given length. It is used at the beginning of each game to ensure the user picks a valid word length.

10. Implement the following method:

```
public int numWordsCurrent()
```

This method returns the number of words in the active list of words. (not the original dictionary.) It is used for debugging purposes to show how many words are left in the active list.

Testing

For this assignment, we do not require specific tests. We would like to see some sort of "testing" in the form of debugging statements to the console. This means that you must use the debug variable provided and, if it is set to true, print out some useful information (whether that be in `prepForRound`, your constructor, `makeGuess`, or all of the methods). It is up to you to decide what debugging output is useful during testing. We expect that you provide sufficient debugging statements. We will not award credit for a single debugging statement.

How the Program Cheats

The Evil Hangman program doesn't actually pick a single word until it is forced to. Suppose that the user has asked for a 5-letter word for a game of Hangman. Instead of picking a specific 5-letter word, the program considers *all* 5-letter words from the original dictionary as potential choices. Then, as the user makes guesses, the program has to somehow fool the user into thinking that it isn't cheating. Your `HangmanManager` object will do this in a very specific way every time the `makeGuess` method is called. Let's look at a small example.

Suppose that the dictionary contains just the following 9 words:

```
[ally, beta, cool, deal, else, flew, good, hope, ibex]
```

Suppose that the user guesses the letter ' e '. You need to indicate which letters in the word you've "picked" are e's. Of course, you haven't picked a word, and so you have multiple options about where you reveal the e's. Every word in the set falls into one of five "word families:"

- "----": which is the pattern for [ally, cool, good]
- "-e--": which is the pattern for [beta, deal]
- "--e-": which is the pattern for [flew, ibex]
- "e--e": which is the pattern for [else]
- "---e": which is the pattern for [hope]

Since the letters you reveal have to correspond to some word in your word list, you can choose to reveal any one of the above five families.

There are many ways to pick which family to reveal - perhaps you want to steer your opponent towards a smaller family with more obscure words, or towards a larger family in the hopes of keeping your options open. In this assignment, for simplicity, we'll adopt the latter approach and always choose the largest of the remaining word families. In this case, it means that you should pick the family "----". This reduces your set of words to:

[ally, cool, good]

Since you didn't reveal any letters (the guess wasn't in the resulting pattern), this will count as a wrong guess.

Let's keep tracing through this example. Given the remaining three-word set, if the user guesses the letter ' o ', then you would break your word list down into two families:

- "-oo-": which is the pattern for [cool, good]
- "----": which is the pattern for [ally]

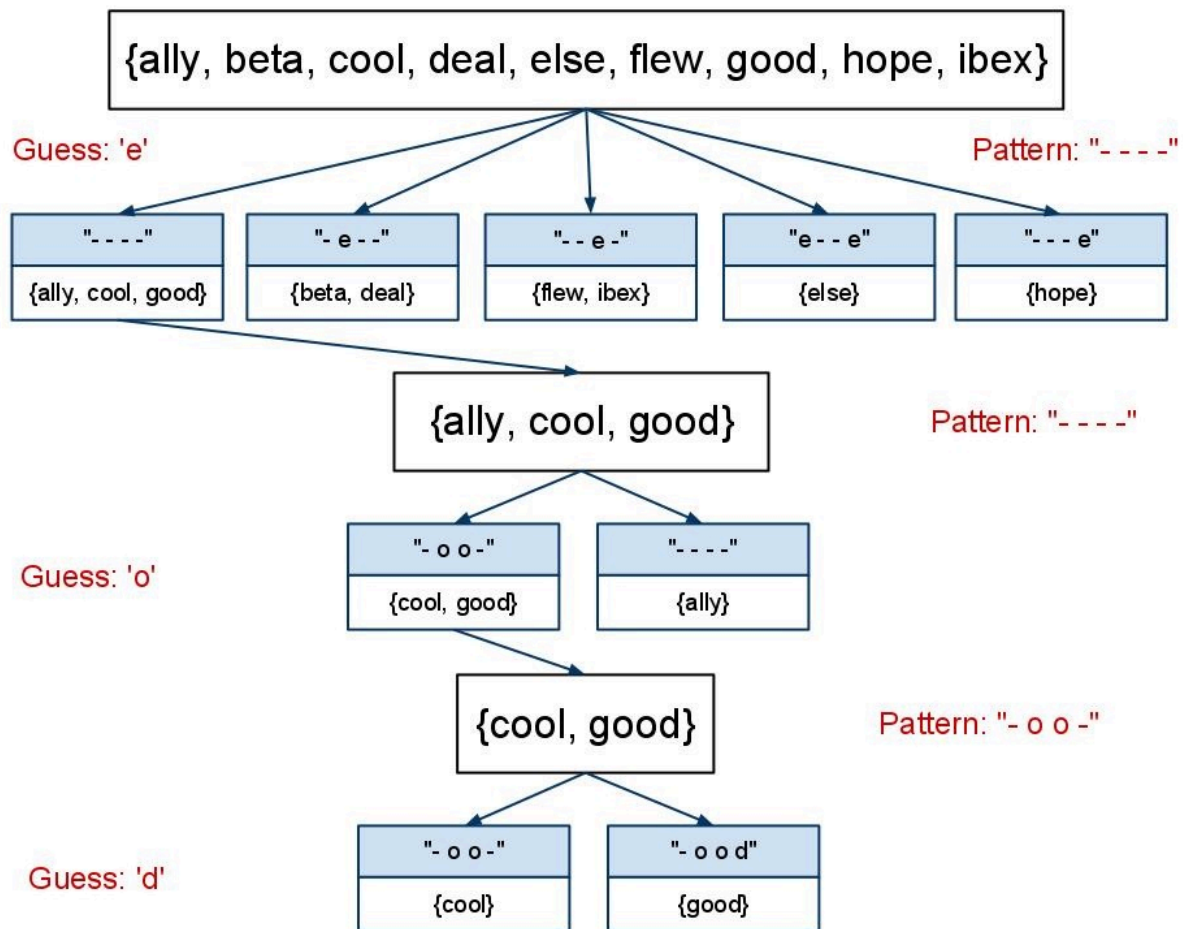
The first of these families is larger than the second, and so you choose it, revealing two ' o 's in the word and reducing your set of words to:

[cool, good]

In this case, the user has correctly guessed a letter because there are two occurrences of ' o ' in the new pattern.

What happens if your opponent guesses a letter that doesn't appear anywhere in your word list? For example, what happens if your opponent now guesses ' t '? This isn't a problem. If you try splitting these words apart into word families, you'll find that there's only one family: the family "-oo-" in which ' t ' appears nowhere, and which contains both "cool" and "good". Since there is only one word family here, it's trivially the largest family, and by picking it you have maintained the word list you already had. You would count this as an incorrect answer.

[Here is a more condensed writeup](#) of the previous example, showing the winnowing process. Here is a diagram that shows the same winnowing process based on user guesses:



[Here is another, more detailed, explanation](#) for how to break the current word list up into families based on the current pattern and current guess from former TA Claire.

You are required to use a map on the assignment. The keys will be the different patterns for each word family. The associated values will be the lists of words that have that pattern.

Professor Mike Scott has made [a helpful video](#) that further explains the algorithm for picking the secret word. **We highly recommend that you watch this video.**

Choosing Difficulty

Part 1: Always *Hardest*

When first coding up `makeGuess`, find all of the word families based on the current list of active words (not the original dictionary, except for the first guess) and always pick the one that is the *hardest*.

Here is how you should determine the *hardest* family:

1. The hardest word list / family is defined as the one with the largest number of words with certain tiebreakers. This family becomes the new list of words for the next round of the game.
2. If there is a tie for the family with the most words (two of the word families are of equal size), pick the family that reveals the fewest total characters.
3. If there is still a tie, pick the family that has the pattern that is "smallest" based on the ASCIIbetical (lexicographical order based on ASCII codes) ordering of the patterns. Look [here](#) for more info on ASCIIbetical order. Here is an example of how lexicographical ordering works:

```
String pat1 = "---a";  
String pat2 = "a---";
```

```
System.out.println(pat1.compareTo(pat2));  
// prints out a negative integer because "---a" is lexicographically  
smaller (or before) "a---"
```

```
System.out.println(pat2.compareTo(pat1));  
// prints out a positive integer because "a---" is lexicographically  
larger (or after) "---a"
```

Part 2: Other Difficulties

In order to moderate the evilness of the program, the user is allowed to enter a difficulty level, (an enumerated type, or enum) from `HangmanDifficulty`. **We recommend that you complete this part of the program last, after you have already completed everything else and thoroughly tested your code.** When starting out, ignore the user's choice for difficulty and implement the algorithm from Part 1. When you have that working, then alter the `makeGuess` method to take into account the selected difficulty.

If the selected difficulty is `HangmanDifficulty.HARD`, the program picks the word list as described above in Part 1. `HangmanDifficulty.HARD` always picks the *hardest* word list / family.

If the difficulty is `HangmanDifficulty.MEDIUM`, the program picks the *hardest* word list **three** times in a row, then the *second hardest* word list **once**, then the *hardest* word list **three** times in a row, then the *second hardest* word list **once**, etc. This pattern repeats until the game is over. If there is only one word list and it is time to pick the *second hardest*, pick the single word list (which would be the *hardest* word list), then continue with the pattern as normal. Here is another view of which word list to pick:

Guess 1: Pick *hardest* word list
Guess 2: Pick *hardest* word list
Guess 3: Pick *hardest* word list
Guess 4: Pick *second hardest* word list if it exists, otherwise, pick *hardest* word list
Future Guesses: Continue with this pattern

If the difficulty is `HangmanDifficulty.EASY`, the program alternates between picking the *hardest* word list and the *second hardest* word list (if it exists). Here is another view of which word list to pick:

Guess 1: Pick *hardest* word list
Guess 2: Pick *second hardest* word list if it exists, otherwise, pick *hardest* word list
Future Guesses: Continue with this pattern

Your solution to this problem must scale if we change which patterns to pick. What if we change the requirement that on easy mode we pick the *4th* hardest pattern every other round and on medium difficulty we pick the *3rd* hardest pattern every fourth round? Your code should scale easily in this case. Hint: There are many ways to find the hardest word list / family and to make the difficulty portion easier. You may find it useful to create new, nested classes that implement the `Comparable` or `Comparator` interfaces.

Submission Checklist

Before you submit, run through our submission checklist to make sure you didn't forget anything! Did you remember to:

1. Implement all the required methods?
2. Ensure your program's output matches the provided sample output?
3. Include helpful and sufficient debugging statements in your code?
4. Fill in the header comment?
5. Follow the [program hygiene guide](#)?
6. Follow the assignment specifications from page 3 of the [Assignment handout](#)?

If you have done all these things, then you're ready to submit `HangmanManager.java` to Gradescope!

Thank you to Professor Mike Scott for creating the previous version of this assignment!

Based on an assignment created by Nick Parlante and Stuart Reges.